

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Corso di preparazione per la selezione territoriale delle Olimpiadi di Informatica

Lezione 4. Ricorsione, divide et impera

Alessio Martino

31 marzo 2022

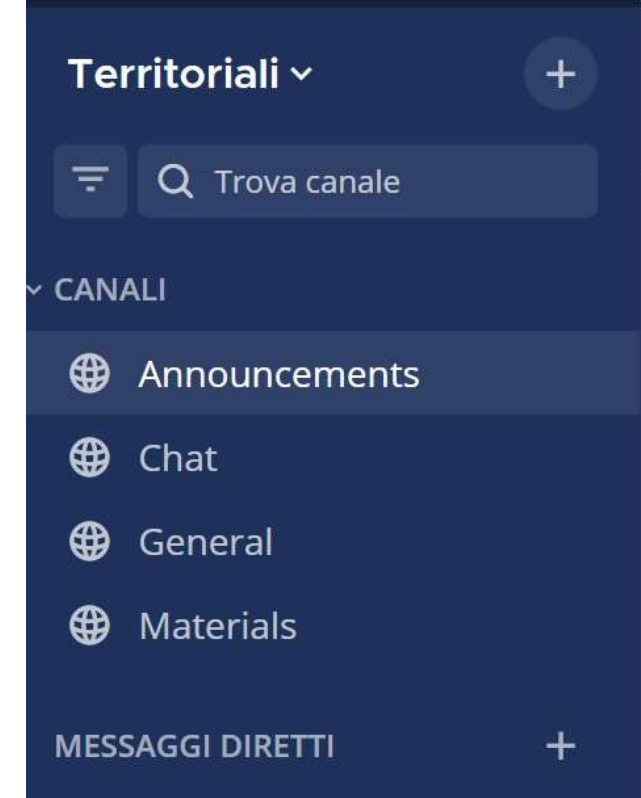
LUISS



Reminder: LUISSTrain

<https://coding.luiss.ml/train/territoriali/>

- 1. *Announcements*** (e.g., link video lezioni)
 - 2. *Chat*** (fai qui le tue domande durante le lezioni)
 - 3. *General*** (discussioni interesse generale)
 - 4. *Materials*** (materiale didattico, slide prima delle lezioni, etc...)
- Controlla i canali: spesso troverai qui le risposte alle tue domande!
 - Non inquinare i canali! Contribuisci a mantenerli puliti e non postare contenuti impropri. Grazie!



Lezioni

Puoi accedere alle lezioni tramite YouTube o Webex:

 **YouTube** (preferibile se hai problemi di connessione)

 **Cisco webex** (limitata a 1000 utenti; però puoi diventare **Chad (Stacy) per un giorno** condividendo la tua soluzione ai problemi che consideriamo di volta in volta!)



Avviso Importante

- (come vi sarete accorti...) dalle scorse lezioni i link Webex e Youtube sono pubblicati **esclusivamente** sul canale Announcement di LuissTrain e **NON** viene più inviata l'email di notifica

Programma di oggi

1. Comunicazioni sul Contest Intermedio (2 aprile)
2. Soluzione di alcuni esercizi della scorsa settimana
3. **Ricorsione**
 1. Esempi
 2. Esercizio
4. **Divide et impera** (divide & conquer)
 1. Esempi
 2. Esercizio
5. Esercizi per casa

Informazioni sul Contest Intermedio

- Il 2 Aprile si terrà il Contest Intermedio
- Il contest sarà aperto tutto il giorno (dalle 00:00 alle 23:59 del 2 Aprile)
- Potete accedere quando preferite. Una volta effettuato il login, avrete 5 ore di tempo per risolvere i problemi
- Il contest si terrà su <https://terry.luiss.ml>
- Per accedere al Contest sarà necessario un token, che potete richiedere scrivendo al bot https://coding.luiss.ml/train/territoriali/messages/@token_bot#
- Per prendere familiarità con la piattaforma di gara, potete provare a risolvere i problemi delle passate edizioni delle Territoriali su <https://territoriali.olinfo.it>
- Anche in questo caso vi sarà richiesto un token, che corrisponde al vostro username su <https://training.olinfo.it>

Soluzioni di alcuni degli esercizi della scorsa settimana



Chad and gigachad is for losers.

Oggi potrete ambire ad essere **terachad**.

Se siete collegati tramite Webex, alzate la mano per presentare le vostre soluzioni.

```
truth = "I am a Chad! "                # or Stacy!  
  
def chad(x):                             # or def stacy(x):  
    return chad(x*(10**3))  
  
chad(truth)
```

don't try this at home, plz

Viaggio in taxi (1/5)

Scrivete in chat per esporre la vostra soluzione:

<https://training.olinfo.it/#/task/taxi/statement>

Finanza creativa (2/5)

Scrivete in chat per esporre la vostra soluzione:

https://training.olinfo.it/#/task/ois_bilancio/statement

Convegno aziendale (3/5)

Scrivete in chat per esporre la vostra soluzione:

https://training.olinfo.it/#/task/mat_convegno/statement

Quantum brackets (4/5)

Scrivete in chat per esporre la vostra soluzione:

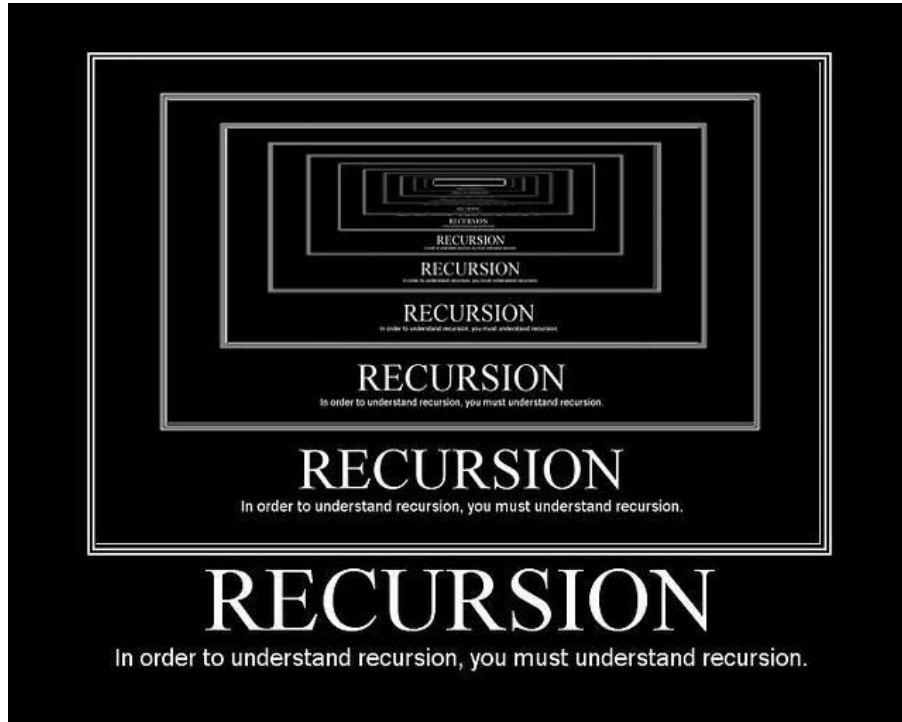
https://training.olinfo.it/#/task/ois_brackets/statement

Accampamento (5/5)

Scrivete in chat per esporre la vostra soluzione:

<https://training.olinfo.it/#/task/padrun/statement>

Ricorsione



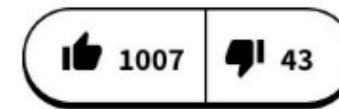
TOP DEFINITION



recursion

[See](#) recursion.

by [Anonymous](#) December 05, 2002

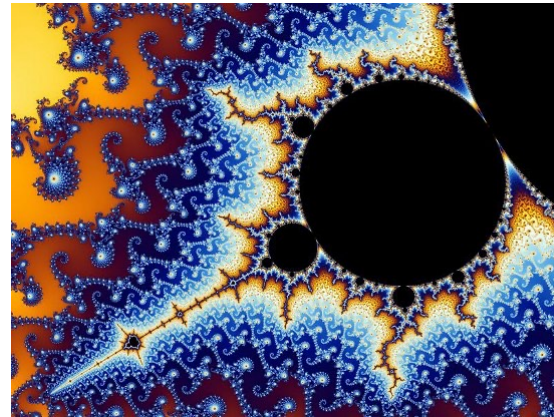
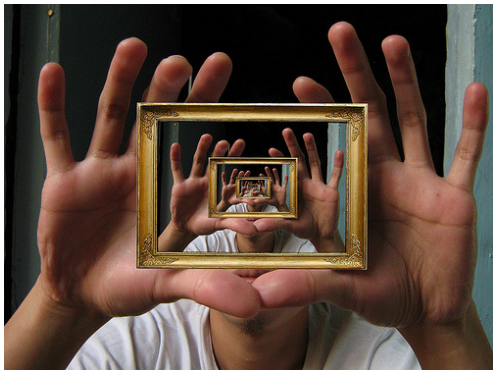


Spot on.

Idea di base

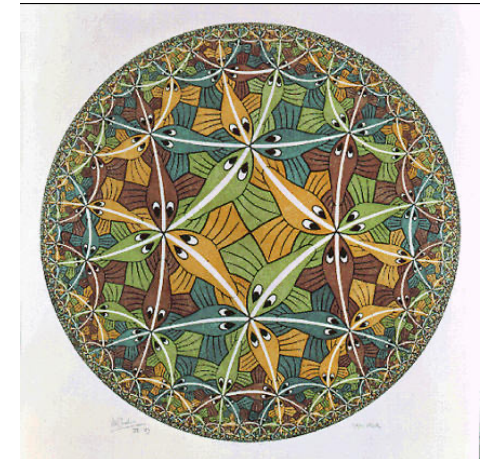
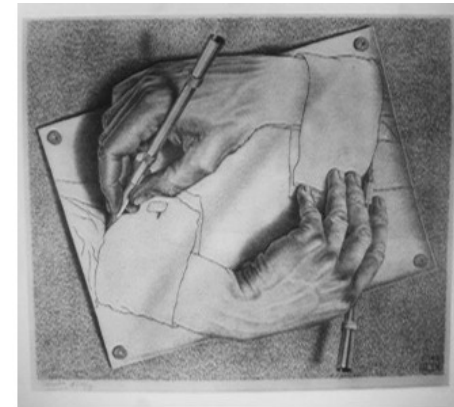
- Concetto semplice, ma molto potente
- Basato sulla nozione di **auto-similarità** e **ripetizione**

Mettete due specchi in
parallelo...



I frattali sono forse la
visualizzazione ricorsiva più
nota (insieme di Mandelbrot)

Anche Escher usava la ricorsione con maestria!



Tool – Lateralus (2001)

1 Black
1 Then
2 White are
3 All I see
5 In my infancy
8 Red and yellow then
came to be
5 Reaching out to me
3 Lets me see

Verse 1

13 As below so above
and beyond I
imagine
8 Drawn beyond the
lines of reason
5 Push the envelope
3 Watch it bend

Pre-chorus



Tool – Lateralus (2001)

Over-thinking, over-
analyzing, separates the
body from the mind

Withering my intu-
ition, missing oppor-
tunities and I must

Feed my will to feel my
moment drawing way out-
side the lines

Chorus

9/8

8/8

7/8

9/8

8/8

7/8

9/8

8/8

7/8

Time signature



987 = Fibonacci(16)

8 = Fibonacci(6)

Tool – Lateralus (2001)

1 Black
1 Then
2 White are
3 All I see
5 In my infancy
8 Red and yellow then
came to be
5 Reaching out to me
3 Lets me see

Verse 1

1 Black
1 Then
2 White are
3 All I see
5 In my infancy
8 Red and yellow then
came to be
5 Reaching out to me
3 Lets me see

Verse 1

2 There is
1 So
1 Much
2 More and
3 Beckons me
5 To look through to
these
8 Infinite
possibilities

Verse 2

Tool – Lateralus (2001)

Fun Nerd fact #1:

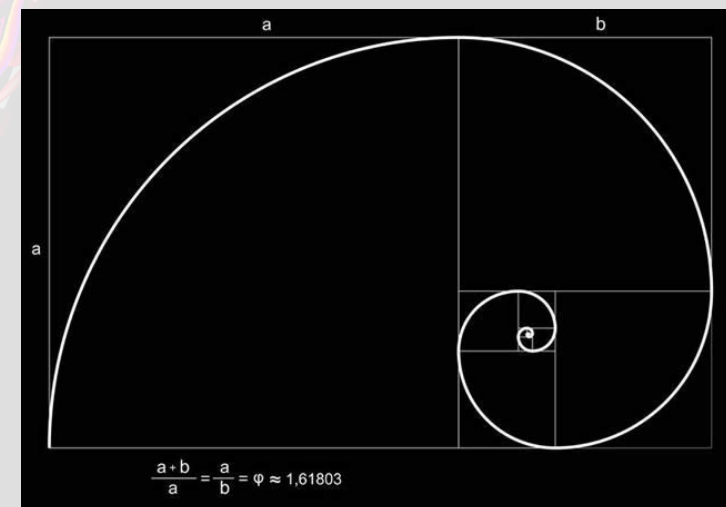
- durata verse 1 + pre-chorus = 55 secondi
- durata verse 1 + verse 2 + pre-chorus = 55 secondi

$$55 = \text{Fibonacci}(10)$$

Fun Nerd fact #2:

- il cantante entra dopo 1:37

$$1:37 = 1.618 \text{ min}$$



Ricorsione e problem solving

- Per usare la ricorsione nel progettare la soluzione di un problema, *cercate di risolvere il problema... assumendo che lo abbiate già risolto!*
- Come è possibile?

Definendo la soluzione di un'istanza del problema in termini della soluzione di istanze più piccole

- La funzione che calcola la soluzione *richiamerà se stessa* (una o più volte) all'interno del proprio corpo
- Spesso si ottiene codice estremamente sintetico ed elegante

Esempio #1: il fattoriale

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1$$

Calcoliamo 5!

$$\begin{aligned} 5! &= 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \\ &\quad \underbrace{\hspace{1.5cm}}_{1!} \\ &\quad \underbrace{\hspace{1.5cm}}_{2! = 1! \cdot 2} \\ &\quad \underbrace{\hspace{1.5cm}}_{3! = 2! \cdot 3} \\ &\quad \underbrace{\hspace{1.5cm}}_{4! = 3! \cdot 4} \\ &\quad \underbrace{\hspace{1.5cm}}_{5! = 4! \cdot 5} \end{aligned}$$

$$\begin{aligned} n! &= (n-1)! \cdot n \\ 0! &= 1 \end{aligned}$$

Esempio #1: il fattoriale

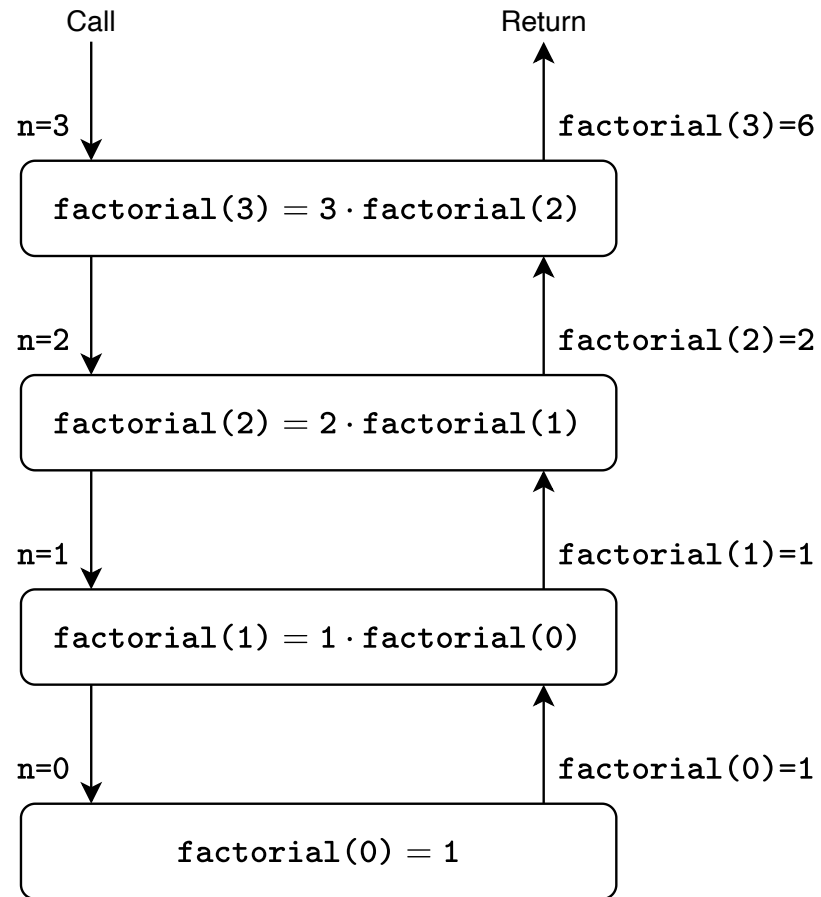


Figura 1: stack diagram per $\text{factorial}(3)$

Gli elementi del procedimento ricorsivo

Per usare un procedimento ricorsivo:

1. Un problema deve essere scomponibile in sottoproblemi **dello stesso tipo**
2. Occorre trovare delle **relazioni** che legano un problema a sottoproblemi simili
3. È necessario conoscere la soluzione di un caso particolare del problema (**caso base** = condizione di terminazione)

Esempio #2: Fibonacci

Definizione. Ogni numero della sequenza di Fibonacci si ottiene sommando i due numeri precedenti: 0, 1, 1, 2, 3, 5, 8, 11, ...

$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{se } n \geq 2 \\ n & \text{se } n = 0, 1 \end{cases}$$

Calcoliamo F_5

$$\begin{array}{c} 5 = 3 + 2 \\ \quad \underbrace{\quad} \quad \underbrace{\quad} \\ \quad 2+1 \quad 1+1 \\ \quad \underbrace{\quad} \\ \quad 1+1 \end{array}$$

Esempio #2: Fibonacci

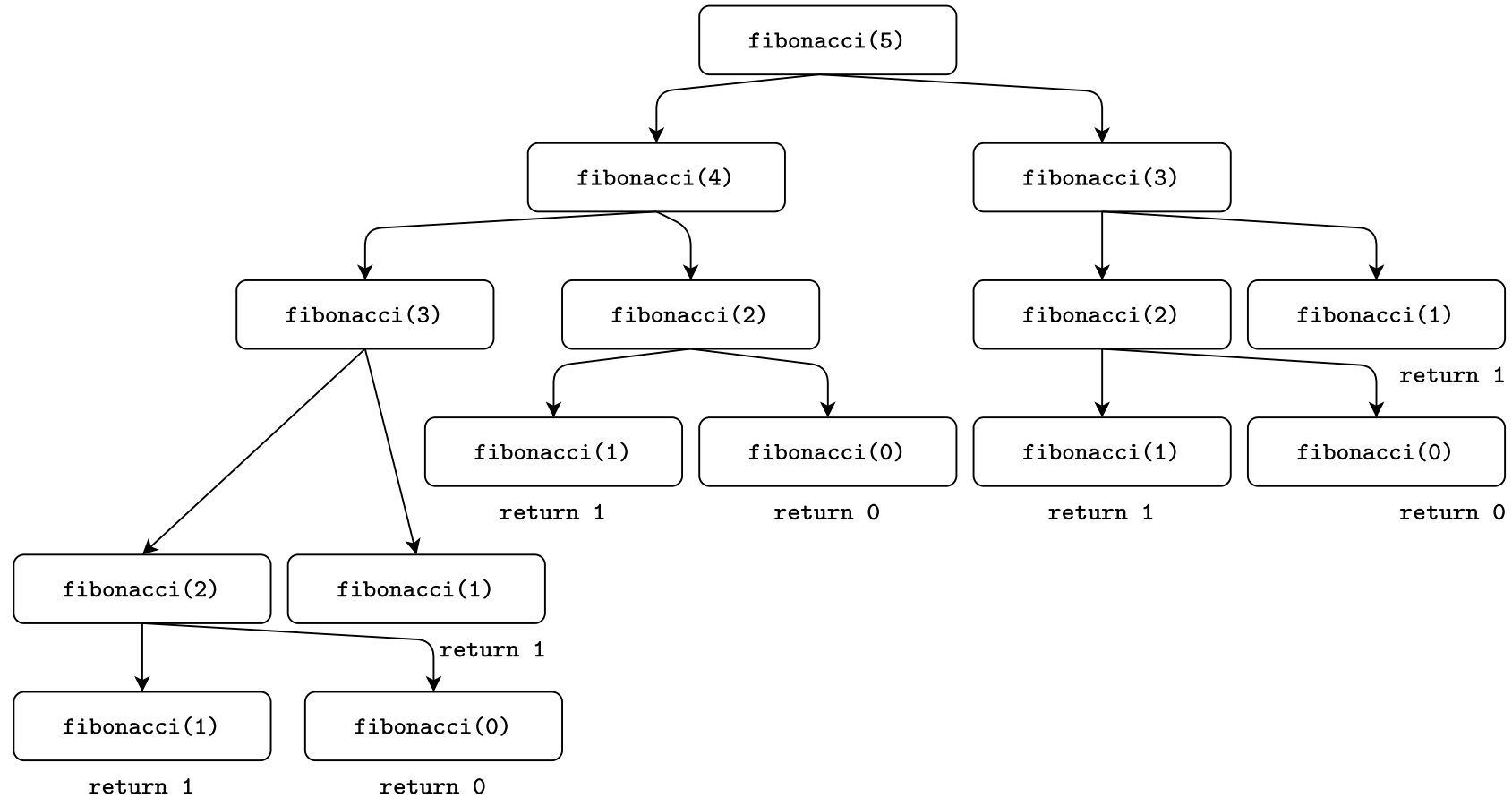


Figura 2: call stack diagram per fibonacci(5)

Esempio #2: Fibonacci

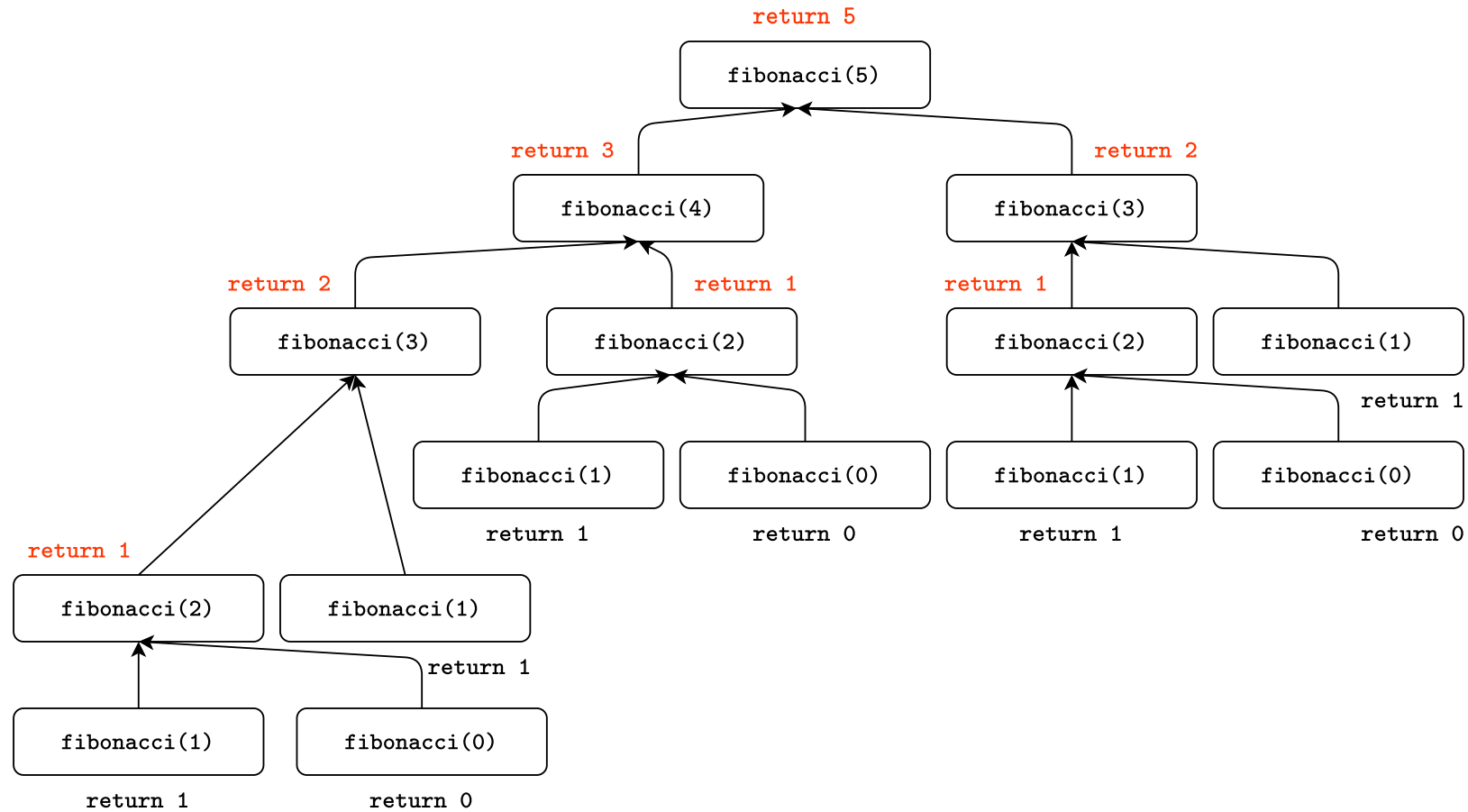


Figura 3: return stack diagram per `fibonacci(5)`

Esempio #2.1: Fibonacci (optimized)

Osservazione: la precedente implementazione di Fibonacci con ricorsione è altamente inefficiente

- per `fibonacci(5)` ho 3 call a `fibonacci(2)` e 2 call a `fibonacci(3)`
- l'albero delle chiamate è un albero binario
- time complexity per F_n : $O(2^n)$

Possiamo fare di meglio?

- sì, se usiamo un approccio iterativo $\rightarrow$ time complexity per F_n : $O(n)$

Possiamo fare di meglio, sempre usando un approccio ricorsivo?

- sì

Esempio #2.1: Fibonacci (optimized)

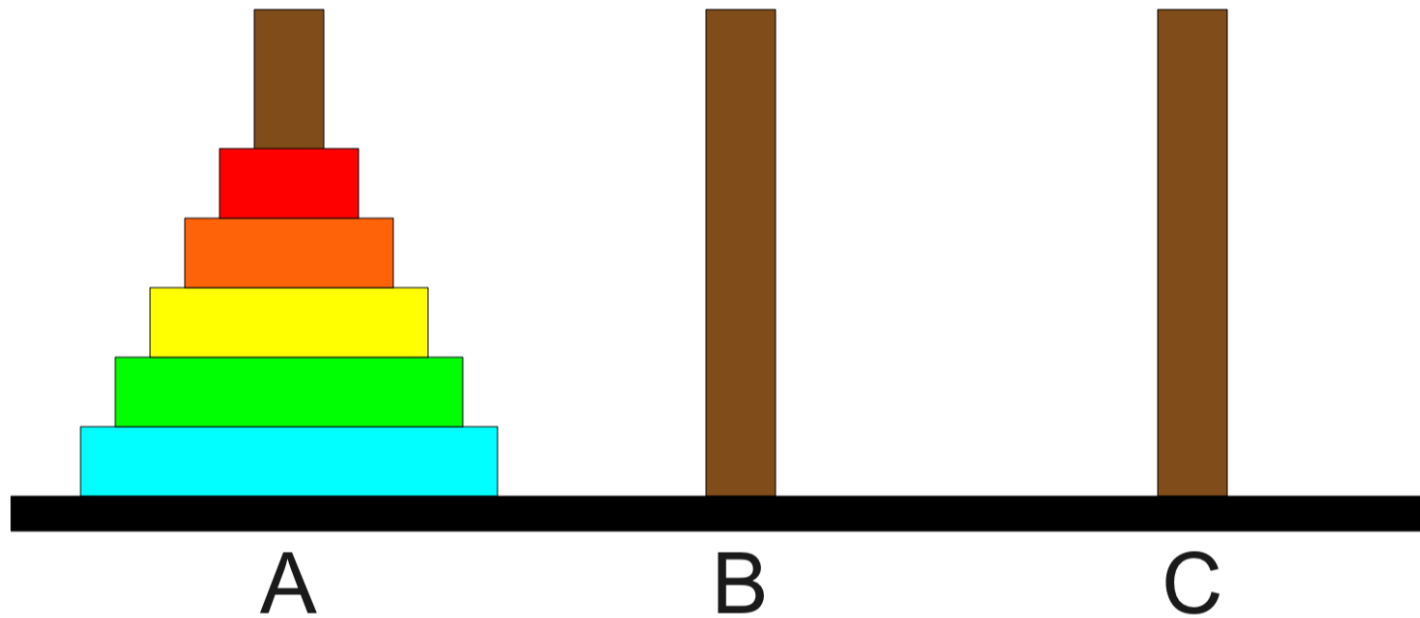
Caching: possiamo salvare i valori di `fibonacci()` già calcolati

- usiamo una *unordered_map* della forma $\{n : F_n\}$
 - la ricerca e l'inserimento in una *unordered_map* hanno complessità $O(1)$
- usiamo una *map* della forma $\{n : F_n\}$
 - la ricerca e l'inserimento in una *map* hanno complessità $O(\log(n))$

Pseudocodice:

```
unsigned long long int fibonacci(int n){  
    se n==0 oppure n==1 → return n;           \\ base case  
    se n è nella map → return map[n];         \\ leggo la cache  
    map[n] = fibonacci(n - 1) + fibonacci(n - 2);  
    return map[n];  
}
```

Esempio #3: le torri di Hanoi



Risolvete lo interattivamente!

<http://dm.compsciclub.ru/app/quiz-hanoi-towers>

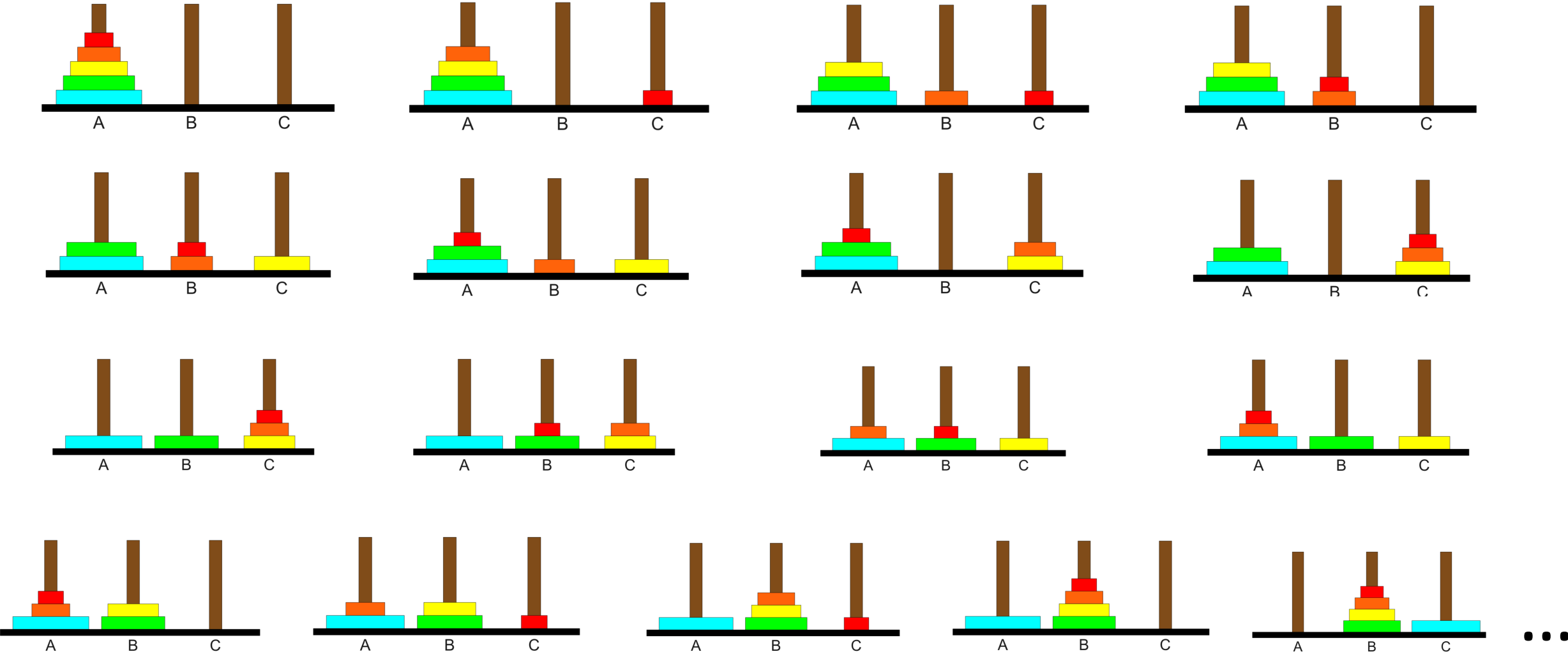
Esempio #3: le torri di Hanoi

Scopo del gioco: spostare il cono di dischi dal paletto di sinistra a quello di destra.

Regole del gioco:

1. si può spostare solo un disco alla volta
2. si può spostare solo il disco che sta sulla sommità della pila di dischi e lo si può sistemare
 - sulla sommità di un'altra pila di dischi, oppure
 - su un paletto vuoto
3. si può mettere un disco solo sopra un disco più grande, mai su uno più piccolo

Soluzione step-by-step

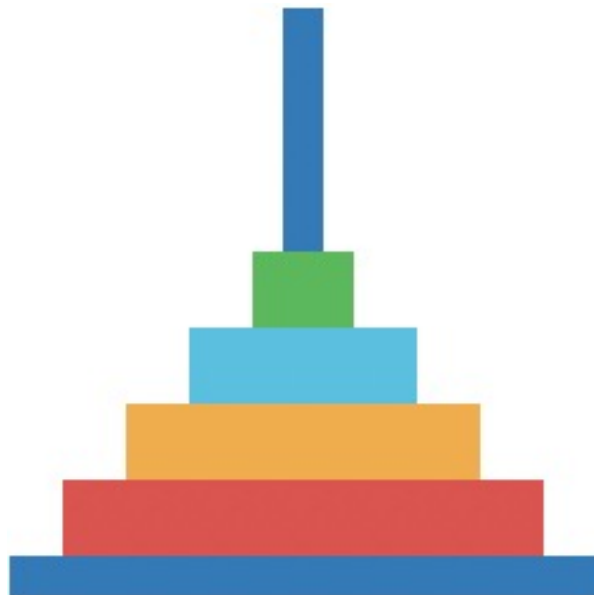


Soluzione ricorsiva

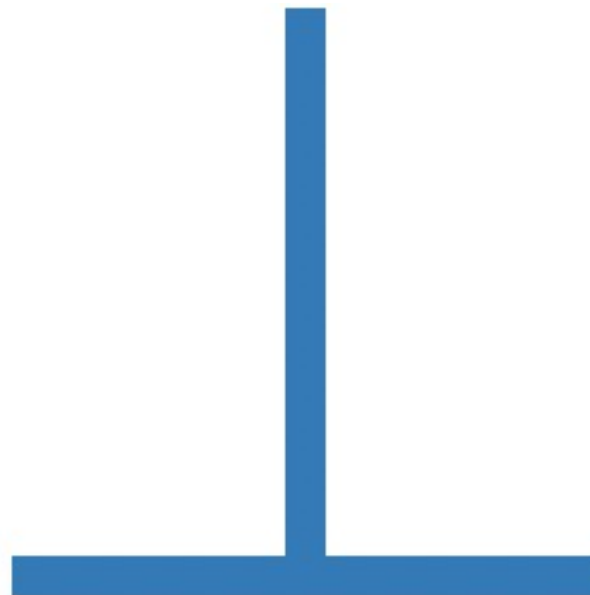
```
towerOfHanoi (int n, char from_stick, char to_stick, char aux_stick)
```

- `n=4`, il numero di dischi
- `from_stick = 'A'`, il paletto di partenza
- `to_stick = 'C'`, il paletto destinazione
- `aux_stick = 'B'`, il paletto ausiliario

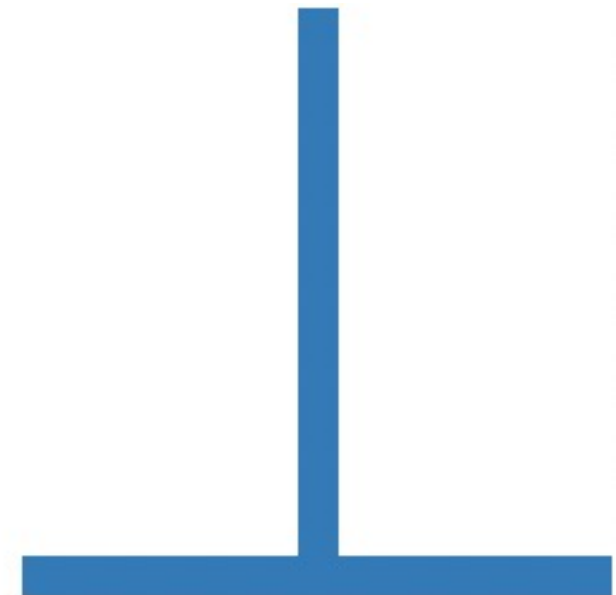
```
towerOfHanoi (4, 'A', 'C', 'B')
```



A



B

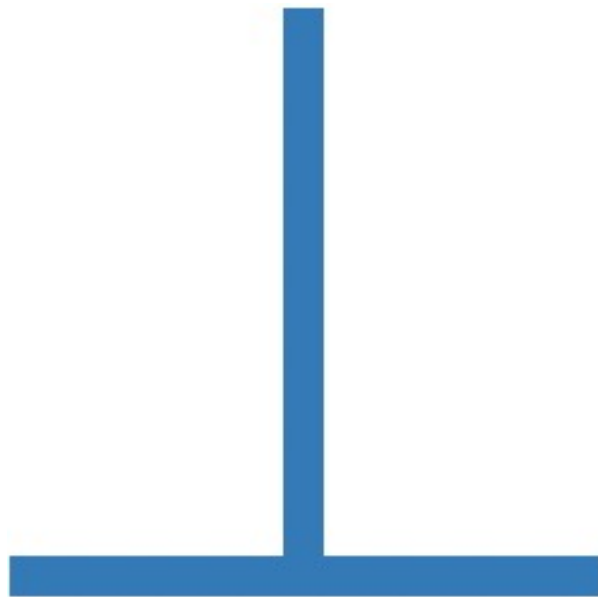


C

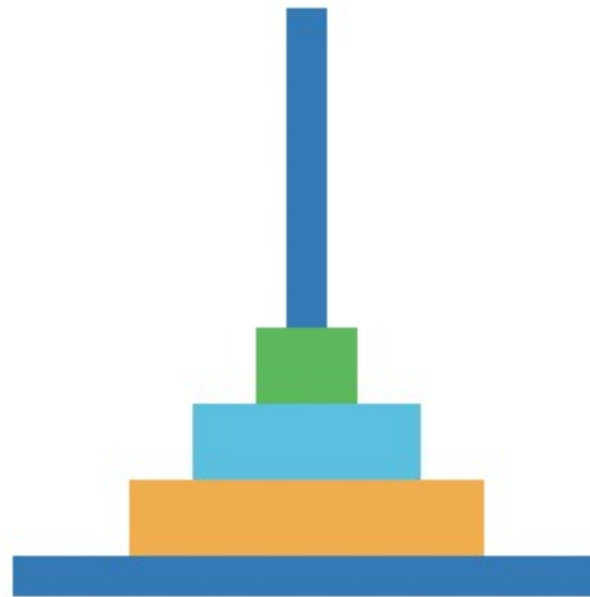
Soluzione ricorsiva

Dividiamo il problema in sotto-problemi:

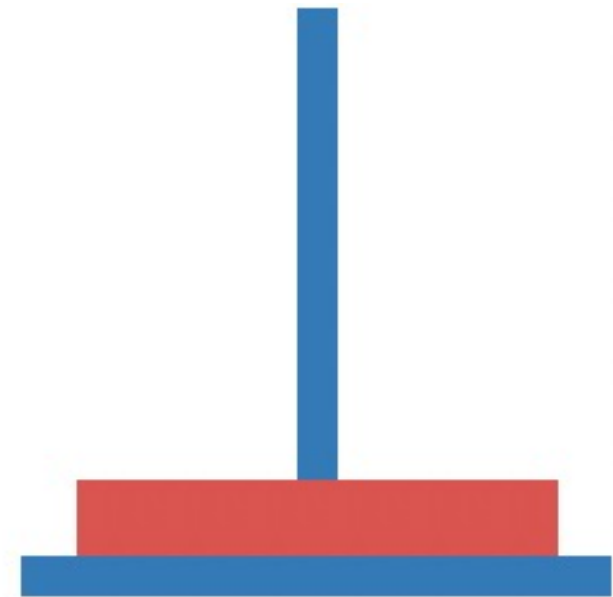
1. Prendere i primi tre dischi e spostarli su B
2. Prendere il disco rosso e spostarlo su C
3. Prendere i tre dischi da B e spostarli su C



A



B

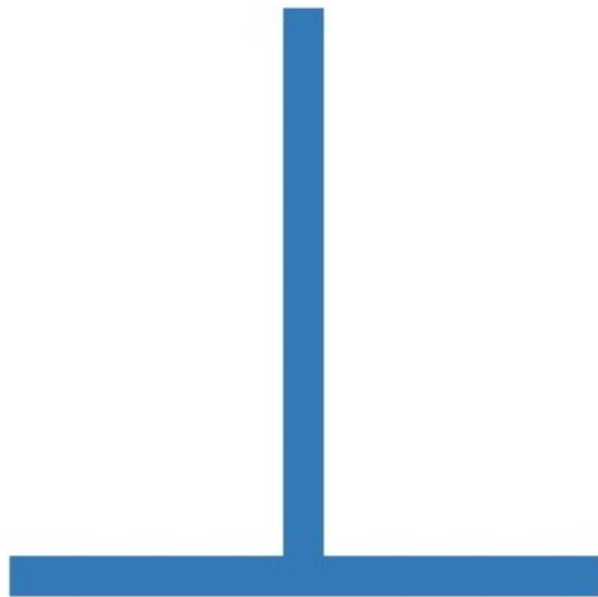


C

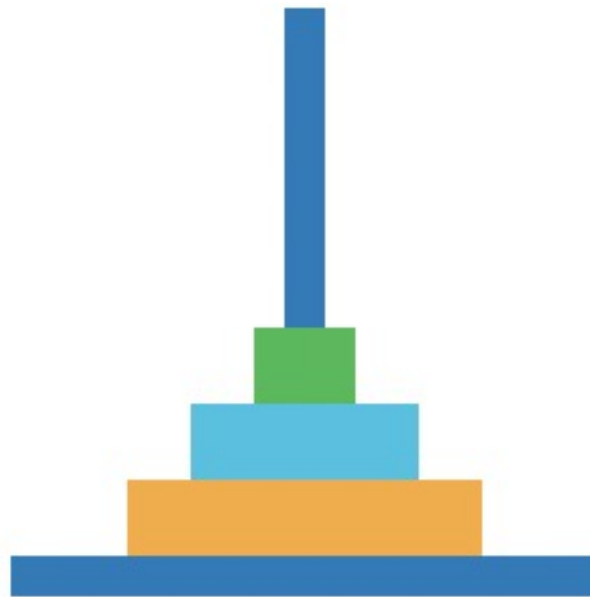
Soluzione ricorsiva

Attenzione:

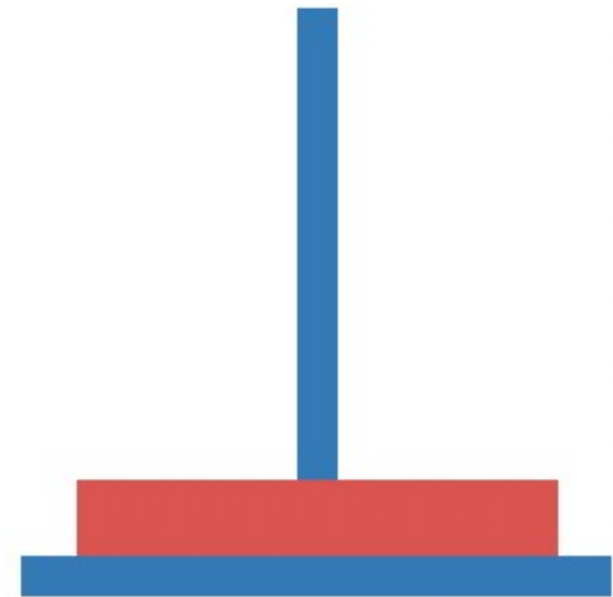
- nello spostare i 4 dischi, ci si pone il problema di spostare 3 dischi!
- spostamento dei tre dischi da A a B e poi da B a C dopo aver mosso il disco rosso
- ... e come faccio a spostare i 3 dischi? ... spostando prima i primi due ... do you see where I'm going?



A



B



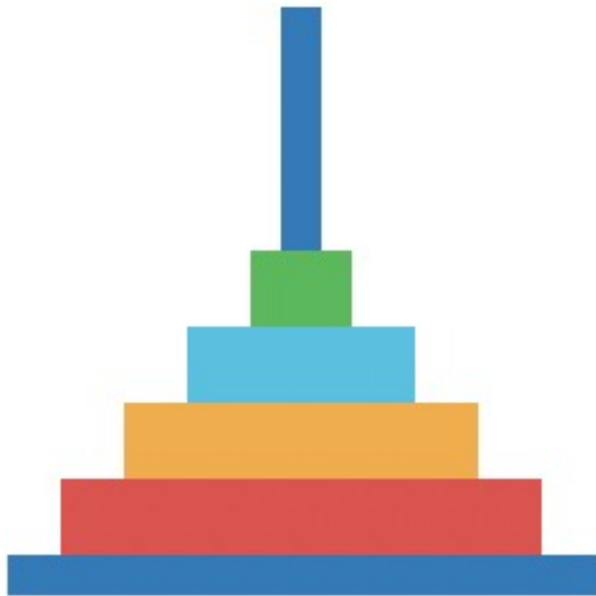
C

Soluzione ricorsiva

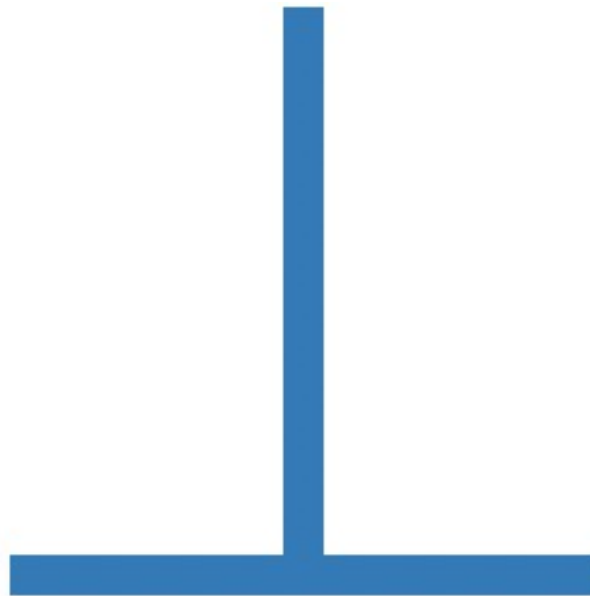
```
towerOfHanoi(4, 'A', 'C', 'B')  
{towerOfHanoi(3, 'A', 'B', 'C')}
```

rosso da A a C

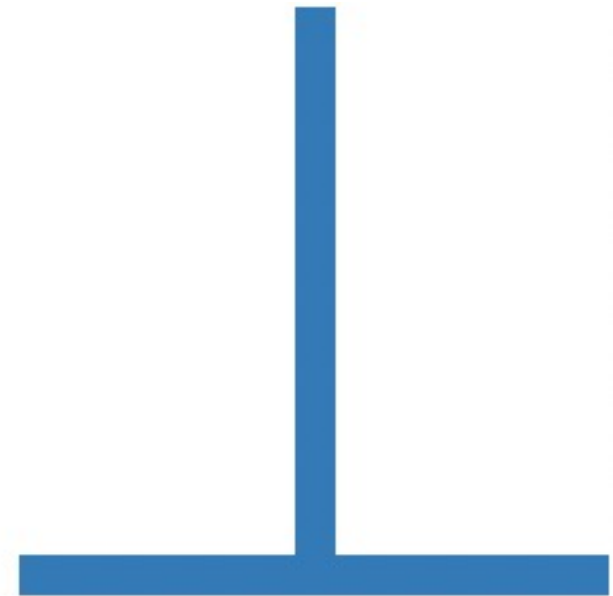
```
towerOfHanoi(3, 'B', 'C', 'A')}
```



A



B



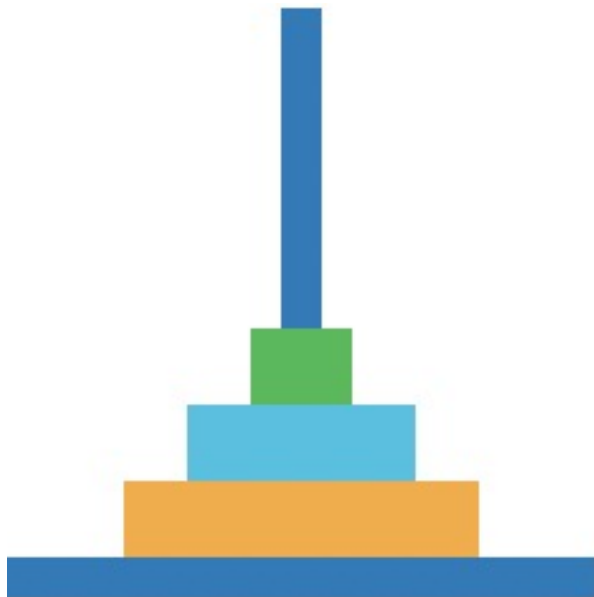
C

Soluzione ricorsiva

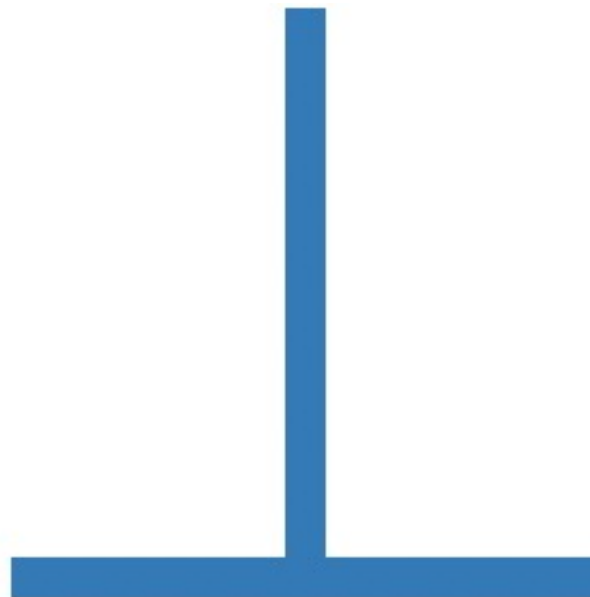
```
towerOfHanoi(4, 'A', 'C', 'B')  
{  
  towerOfHanoi(3, 'A', 'B', 'C')  
  {  
    towerOfHanoi(2, 'A', 'C', 'B')  }}
```

rosso da A a C
arancio da A a B

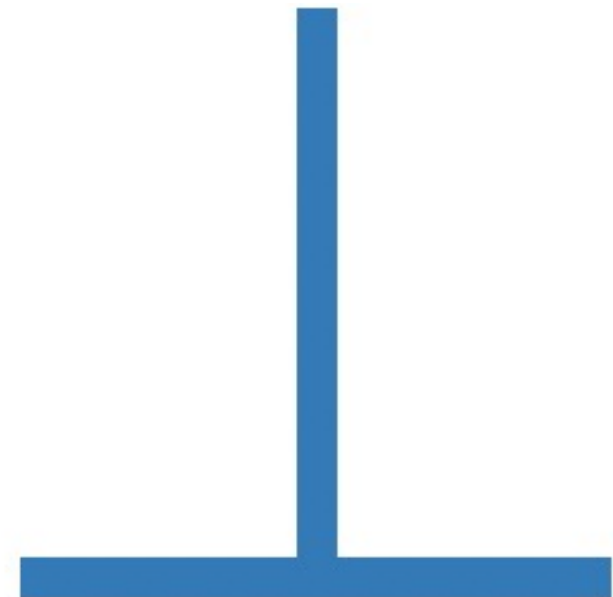
```
towerOfHanoi(3, 'B', 'C', 'A')}  
towerOfHanoi(2, 'C', 'B', 'A')
```



A



B



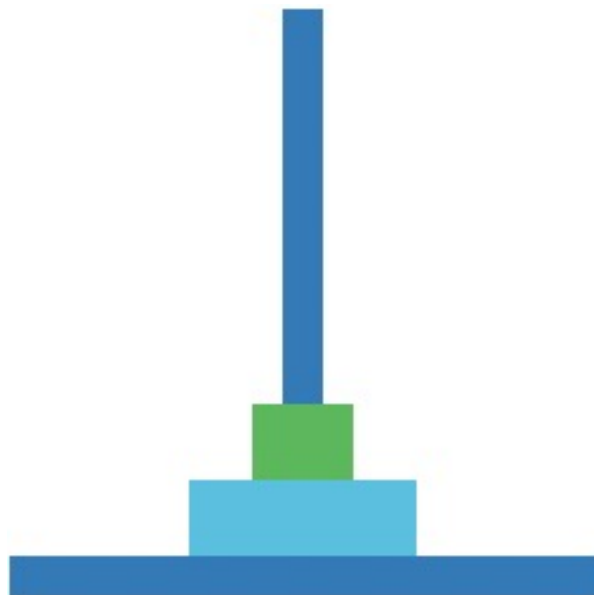
C

Soluzione ricorsiva

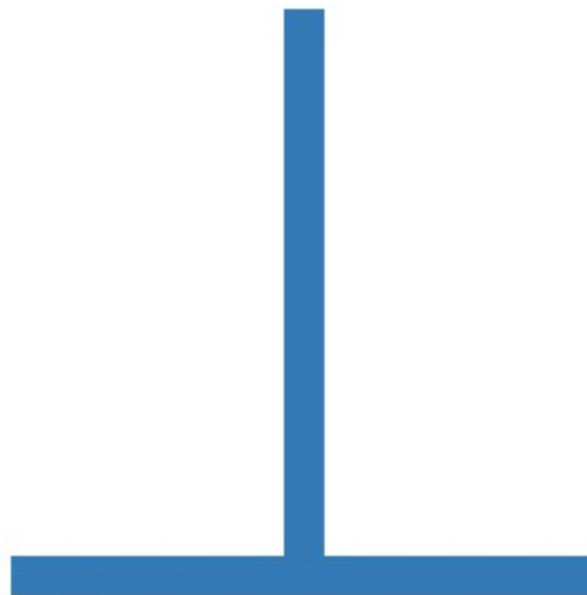
```
towerOfHanoi(4, 'A', 'C', 'B')  
{  
  towerOfHanoi(3, 'A', 'B', 'C')  
  {  
    towerOfHanoi(2, 'A', 'C', 'B')  
    {  
      towerOfHanoi(1, 'A', 'B', 'C')    }}
```

rosso in C
arancio in B
azzurro in C

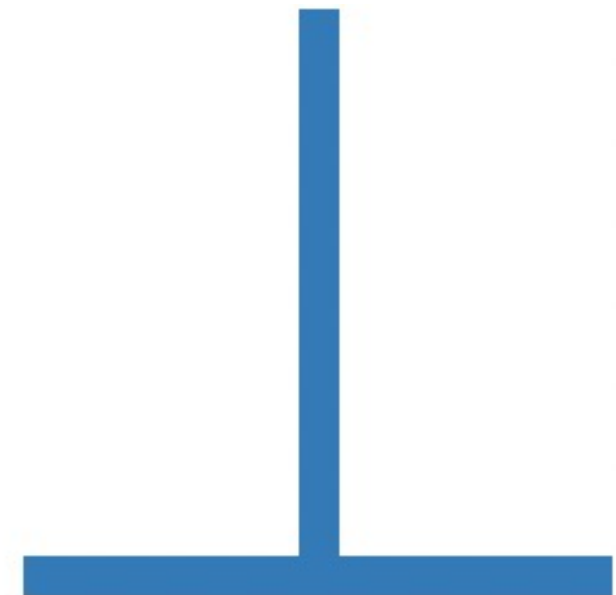
```
towerOfHanoi(3, 'B', 'C', 'A')  
towerOfHanoi(2, 'C', 'B', 'A')  
towerOfHanoi(1, 'B', 'C', 'A')
```



A



B

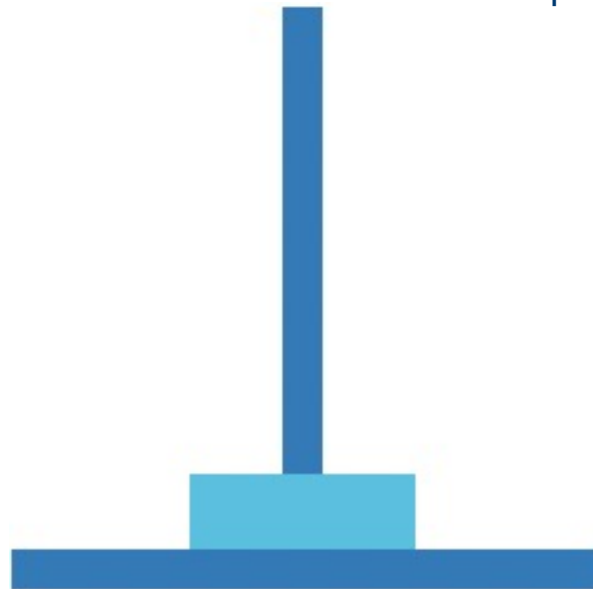


C

Soluzione ricorsiva

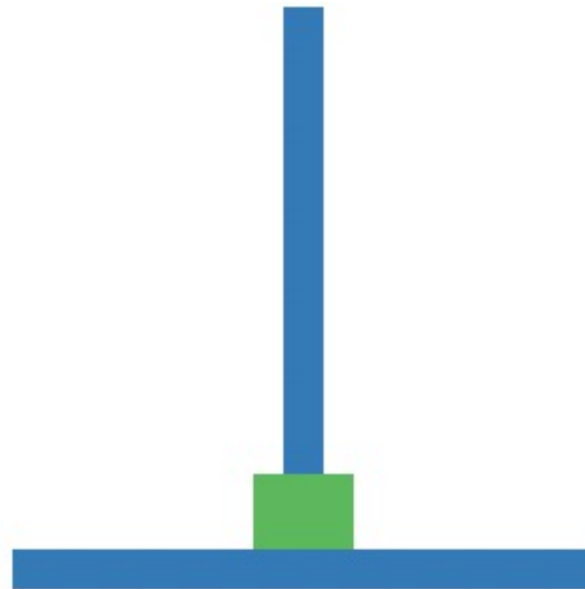
```
towerOfHanoi(4, 'A', 'C', 'B')  
{  
  towerOfHanoi(3, 'A', 'B', 'C')  
  {  
    towerOfHanoi(2, 'A', 'C', 'B')  
    {  
      towerOfHanoi(1, 'A', 'B', 'C')}
```

Abbiamo un solo disco. Prendiamo e spostiamolo.



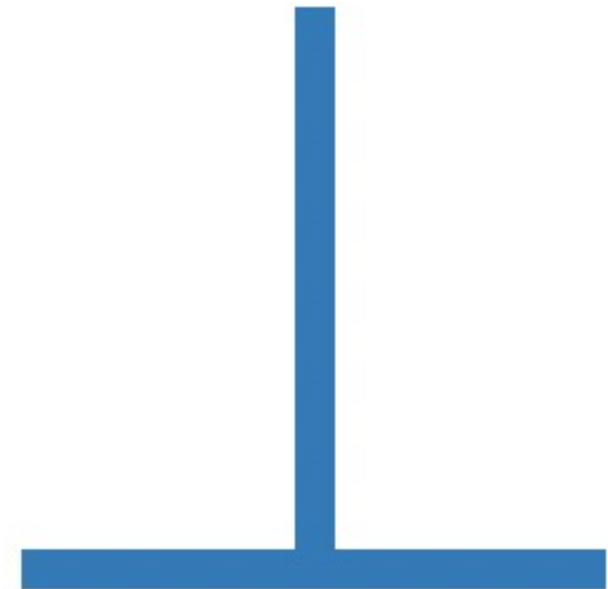
A

rosso in C
arancio in B
azzurro in C



B

```
towerOfHanoi(3, 'B', 'C', 'A')  
towerOfHanoi(2, 'C', 'B', 'A')  
towerOfHanoi(1, 'B', 'C', 'A')
```



C

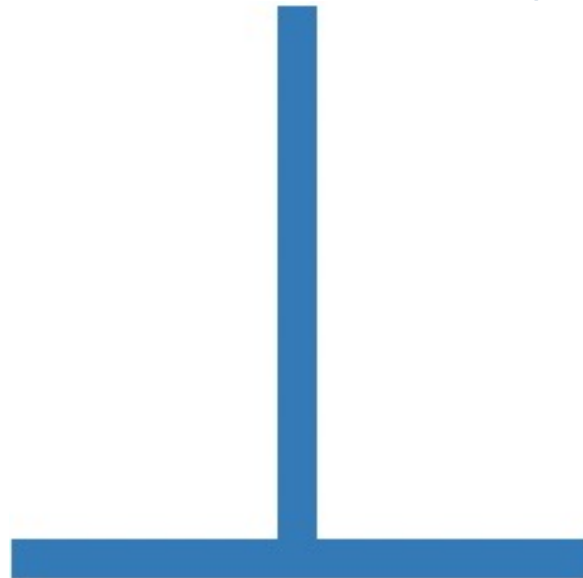
Soluzione ricorsiva

```
towerOfHanoi(4, 'A', 'C', 'B')  
{towerOfHanoi(3, 'A', 'B', 'C')  
  {towerOfHanoi(2, 'A', 'C', 'B')  
    {towerOfHanoi(1, 'A', 'B', 'C')}
```

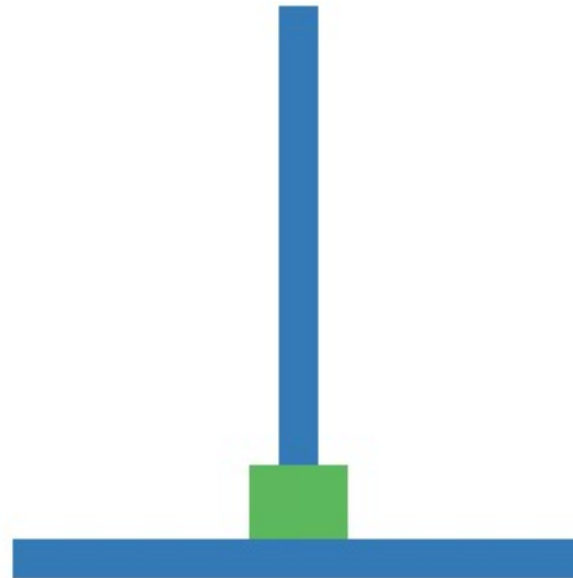
rosso in C
arancio in B
azzurro in C

```
towerOfHanoi(3, 'B', 'C', 'A')}  
towerOfHanoi(2, 'C', 'B', 'A')}  
towerOfHanoi(1, 'B', 'C', 'A')}
```

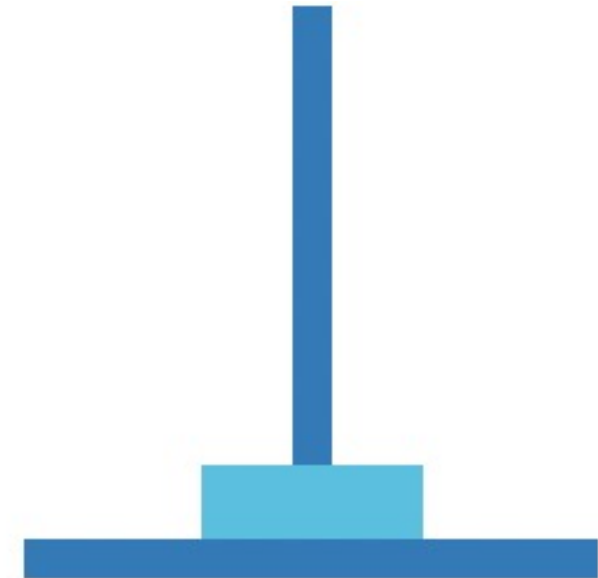
Abbiamo un solo disco. Prendiamo e spostiamolo.



A



B



C

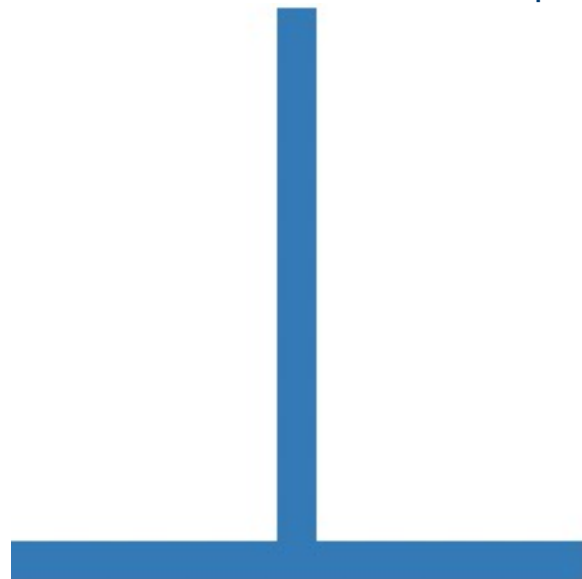
Soluzione ricorsiva

```
towerOfHanoi(4, 'A', 'C', 'B')  
{towerOfHanoi(3, 'A', 'B', 'C')  
  {towerOfHanoi(2, 'A', 'C', 'B')  
    {towerOfHanoi(1, 'A', 'B', 'C')}
```

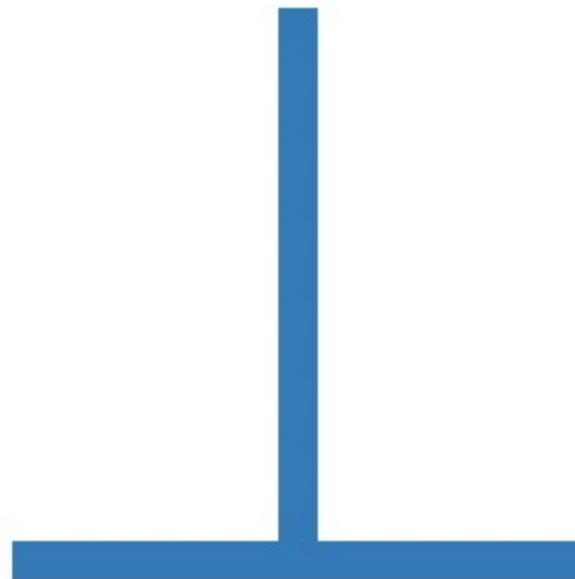
rosso in C
arancio in B
azzurro in C

```
towerOfHanoi(3, 'B', 'C', 'A')}  
towerOfHanoi(2, 'C', 'B', 'A')}  
towerOfHanoi(1, 'B', 'C', 'A')}
```

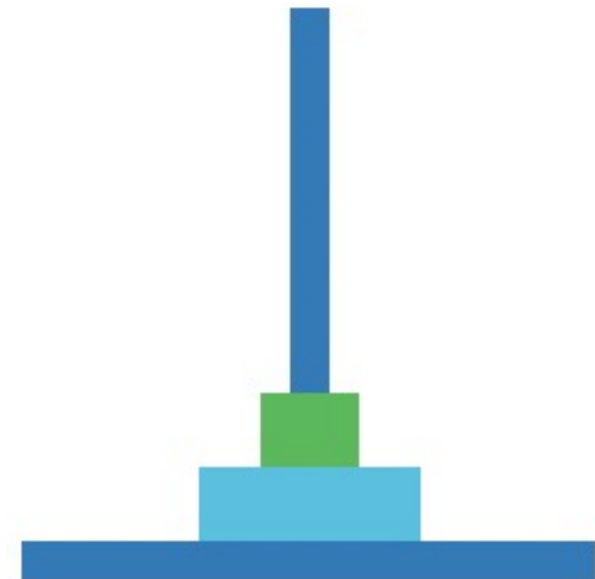
Abbiamo un solo disco. Prendiamo e spostiamolo.



A



B



C

Soluzione ricorsiva

```
towerOfHanoi(4, 'A', 'C', 'B')  
{towerOfHanoi(3, 'A', 'B', 'C')  
  {towerOfHanoi(2, 'A', 'C', 'B')}
```

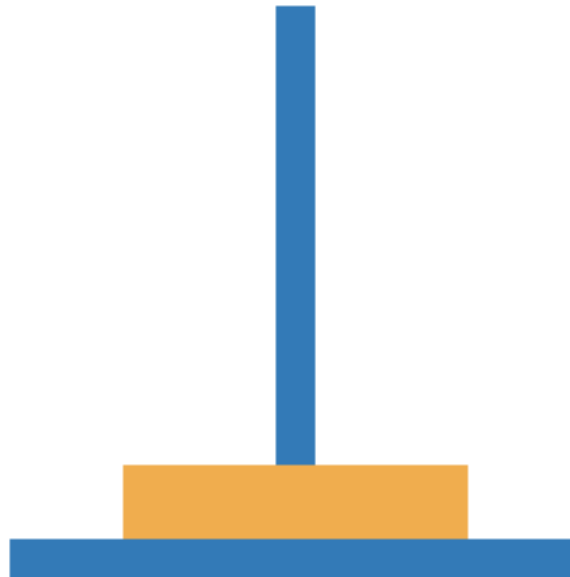
rosso in C
arancio in B

```
towerOfHanoi(3, 'B', 'C', 'A')}  
towerOfHanoi(2, 'C', 'B', 'A')}
```

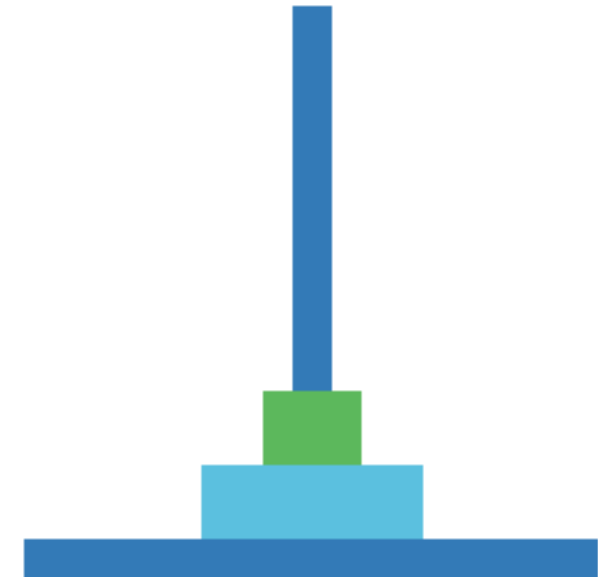
Abbiamo un solo disco. Prendiamo e spostiamolo.



A



B



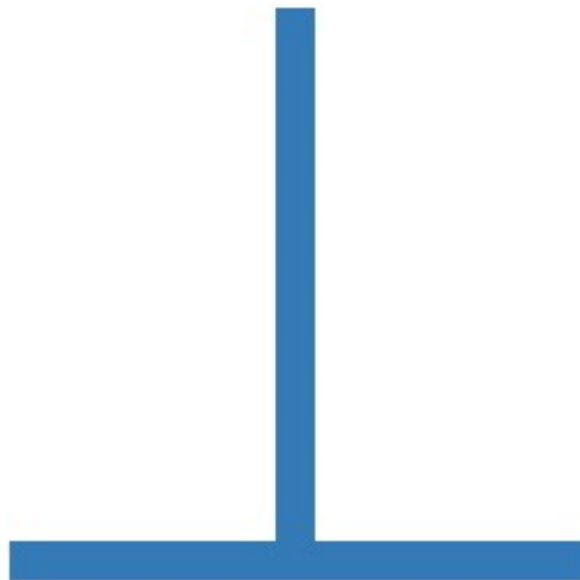
C

Soluzione ricorsiva

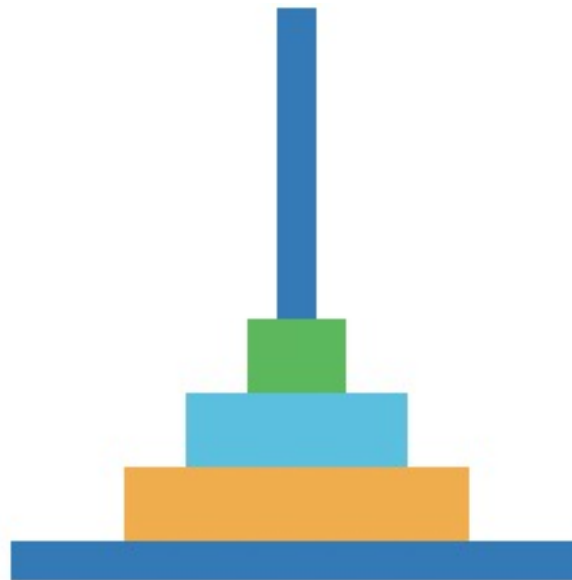
```
towerOfHanoi(4, 'A', 'C', 'B')  
{towerOfHanoi(3, 'A', 'B', 'C')  
  {towerOfHanoi(2, 'A', 'C', 'B')}
```

rosso in C
arancio in B

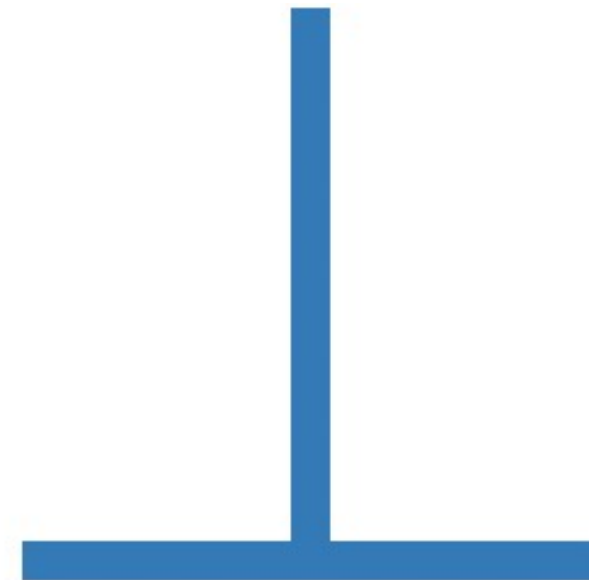
```
towerOfHanoi(3, 'B', 'C', 'A')}  
towerOfHanoi(2, 'C', 'B', 'A')}
```



A



B



C

Soluzione ricorsiva

```
towerOfHanoi(4, 'A', 'C', 'B')  
{towerOfHanoi(3, 'A', 'B', 'C')}
```

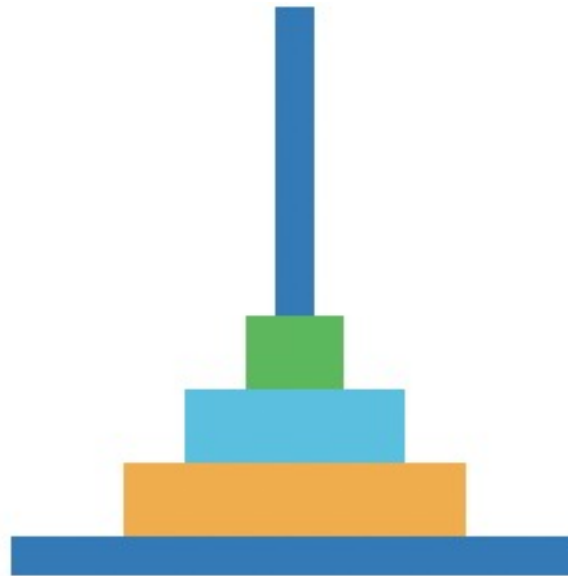
rosso in C

```
towerOfHanoi(3, 'B', 'C', 'A')}
```

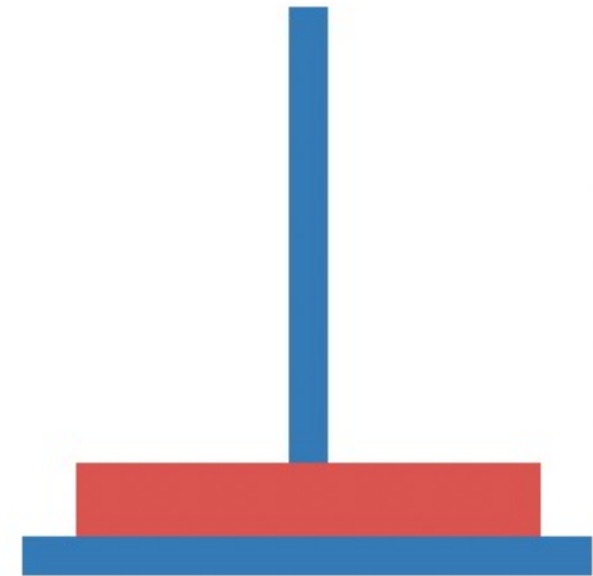
Abbiamo un solo disco. Prendiamo e spostiamolo.



A



B



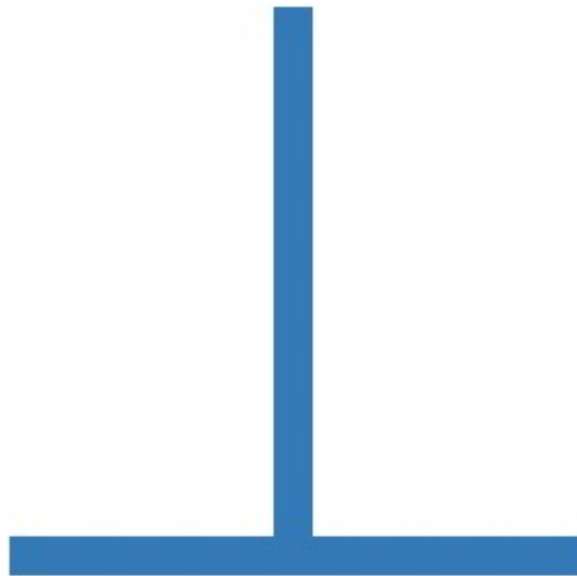
C

Soluzione ricorsiva

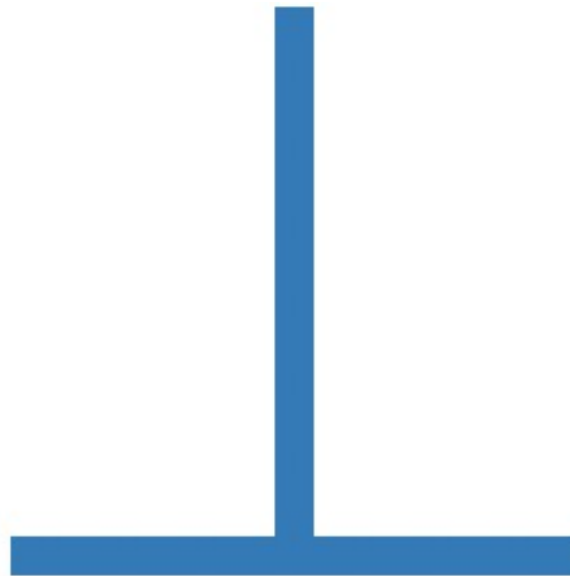
```
towerOfHanoi(4, 'A', 'C', 'B')  
{towerOfHanoi(3, 'A', 'B', 'C')}
```

rosso in C

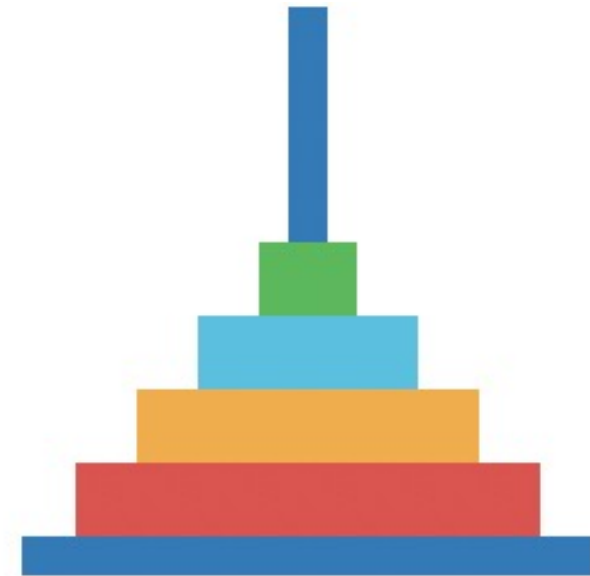
towerOfHanoi(3, 'B', 'C', 'A')}



A



B



C

Un'implementazione elegante (e sintetica!)

```
void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod)
{
    if (n == 0){
        return;
    }
    towerOfHanoi(n-1, from_rod, aux_rod, to_rod);
    cout << "Move disk " << n << " from rod " << from_rod <<
        " to rod " << to_rod << endl;
    towerOfHanoi(n-1, aux_rod, to_rod, from_rod);
}
```

Quando usare la ricorsione?

- Problemi di tipo “**divide et impera**” (next): la soluzione si ottiene suddividendo il problema in due o più sottoproblemi simili, che vengono risolti separatamente ricombinando poi le soluzioni per ottenere la soluzione al problema di partenza
- Soluzioni “**a forza bruta**”: per generare tutti i casi possibili tramite **backtracking** (ad esempio, tutti i sottoinsiemi, tutte le permutazioni...) e scegliere il caso ottimo per il problema in questione
- Problemi di **programmazione dinamica** (lezione del 7-14 aprile)

Problemi con la ricorsione

1. Ricorsione infinita

- Cosa accade se manca il caso base o se la funzione viene richiamata con parametri non corretti?
- Ad esempio, cosa restituisce `factorial(-1)`?

2. Non è immediato calcolare il tempo di esecuzione di funzioni ricorsive

- Si richiedono tecniche matematiche sofisticate (equazioni di ricorrenza)

3. Siate cauti con le chiamate ricorsive: fare più di una chiamata potrebbe far aumentare di molto i tempi di esecuzione (anche in modo esponenziale, e.g., 2^n). Un esempio? Fibonacci!

- Valutate sempre i vincoli del problema (dimensione delle istanze di input)

Non tutto ciò che è intervallo, è greedy.

Missioni Segrete (Territoriali 2008)

<https://training.olinfo.it/#/task/missioni/statement>



Il Commissario Basettoni ha presentato a Topolino le missioni che egli dovrà svolgere segretamente nel corso dell'anno. Per ogni missione, oltre al luogo da raggiungere, Basettoni ne indica la durata in giorni e la data massima entro cui deve essere completata. In altri termini, la missione può iniziare in qualunque giorno dell'anno ma deve durare esattamente il numero di giorni indicato e terminare non oltre la data di scadenza.

Topolino, presa la lista delle missioni ricevuta da Basettoni, ordina tali missioni in base alla loro data di scadenza. Quindi, numera i giorni dell'anno da 1 a 365 (non esistono anni bisestili a Topolinia) e trasforma le date di scadenza in numeri secondo tale numerazione. Per esempio, se una missione dura 15 giorni e deve essere svolta entro il 18 febbraio, Topolino la vede semplicemente come una coppia di interi 15 49 (in quanto il 18 febbraio è il quarantanovesimo giorno dell'anno).

Poiché può svolgere una sola missione alla volta, Topolino sa che potrebbe svolgerne solo alcune pur iniziando una missione il giorno immediatamente successivo a quello in cui termina la precedente missione. Vuole perciò sapere il numero massimo di missioni che è in grado di svolgere rispettando i vincoli sulla loro durata e scadenza. Supponendo che Topolino già fornisca le coppie di interi ordinate per scadenza (il secondo membro delle coppie), aiutatelo a calcolare il massimo numero di missioni che può svolgere.

Per esempio, se ci sono quattro missioni, una di tre giorni da terminare entro il 5 gennaio, una di quattro giorni entro l'8 gennaio, una di tre giorni entro il 9 gennaio e una di 6 giorni entro il 12 gennaio, Topolino vi fornisce la lista di quattro coppie 3 5, 4 8, 3 9 e 6 12. Il numero massimo di missioni che può svolgere è pari a tre, ossia le missioni corrispondenti alle coppie 3 5, 3 9 e 6 12: la prima missione inizia il primo di gennaio e termina il 3 gennaio; la seconda inizia il 4 gennaio e termina il 6 gennaio; la terza inizia il 7 gennaio e termina il 12 gennaio. (Notare che, scegliendo la missione corrispondente alla coppia 4 8, Topolino può svolgere al più due missioni.)

Dati di input

Il file `input.txt` è composto da $N+1$ righe. La prima riga contiene un intero positivo che rappresenta il numero N di missioni presentate da Basettoni a Topolino.

Le successive N righe rappresentano durata e scadenza delle missioni: ciascuna riga è composta da due interi d e s separati da uno spazio, a rappresentare che la corrispondente missione dura d giorni e deve essere completata entro l' s -esimo giorno dell'anno.

Dati di output

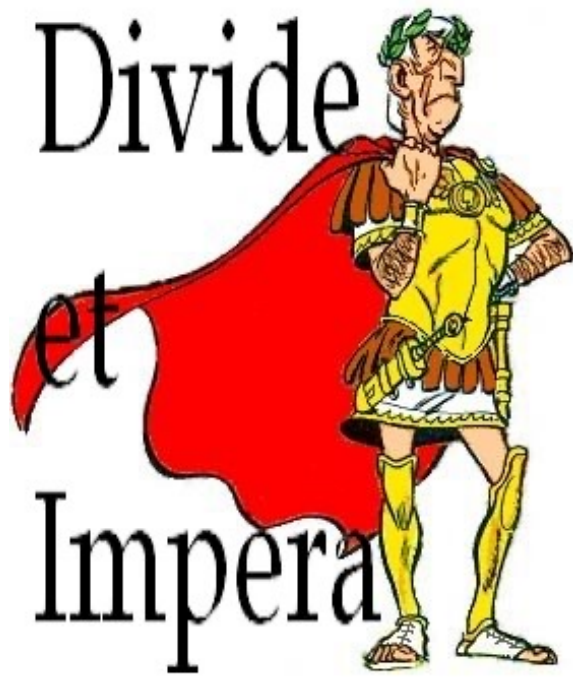
Il file `output.txt` è composto da una sola riga contenente un intero che rappresenta il massimo numero di missioni che Topolino può svolgere rispettando i vincoli su durata e scadenza.

Esempi di input/output

File <code>input.txt</code>	File <code>output.txt</code>
4 3 5 4 8 3 9 6 12	3

Divide et impera

in inglese, *divide & conquer*



Approccio

Struttura ricorsiva:

1. **Dividere** il problema in sottoproblemi simili all'originale, ma più piccoli
2. **Conquistare** i sottoproblemi risolvendoli ricorsivamente. Se sono abbastanza piccoli, risolverli in modo diretto
3. **Combinare** le soluzioni dei sottoproblemi per ottenere la soluzione del problema originale

Esempio #1: MergeSort

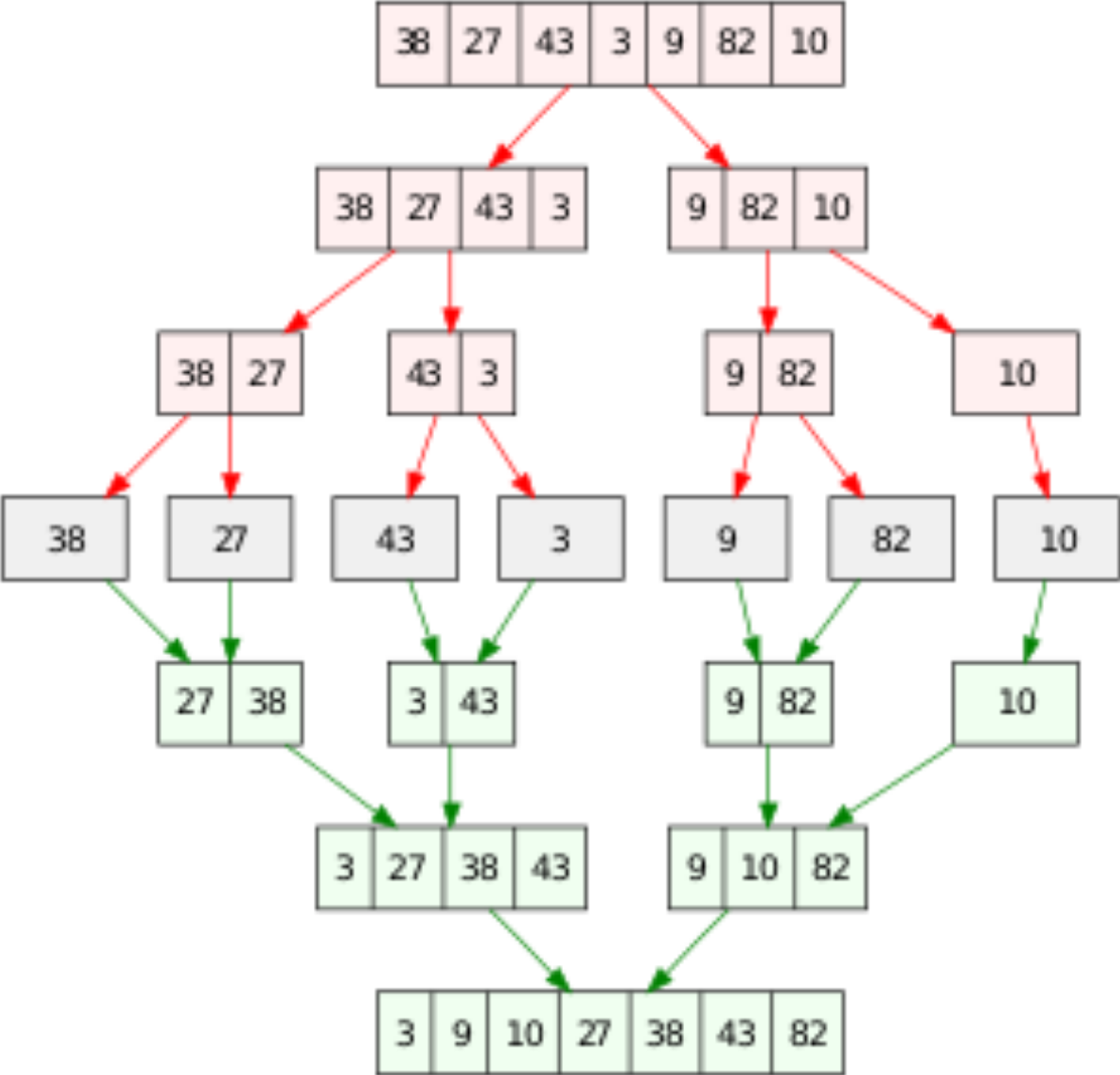
Il problema: ordinare una sequenza di n elementi

L'algoritmo:

1. **Dividere** la sequenza lunga n in due sottosequenze lunghe $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$
2. **Ordinare** le due sottosequenze ricorsivamente usando mergesort (a meno che non abbiano un solo elemento)
3. **Combinare** le soluzioni **fondendo** le due sottosequenze ordinate in una sequenza ordinata lunga n

Nota: la fusione di due sequenze ordinate è un problema facile che può essere risolto in tempo lineare

Graficamente



Implementazione

```
/* s e d sono gli indici sinistro e destro del
   sottoarray da ordinare */
void mergeSort(int arr[], int s, int d) {
    if (s < d) {
        int m = (s+d)/2;           // divide
        mergeSort(arr, s, m);      // conquer
        mergeSort(arr, m+1, d);    // conquer
        fondi(arr, s, m, d);       // combine
    }
}
```

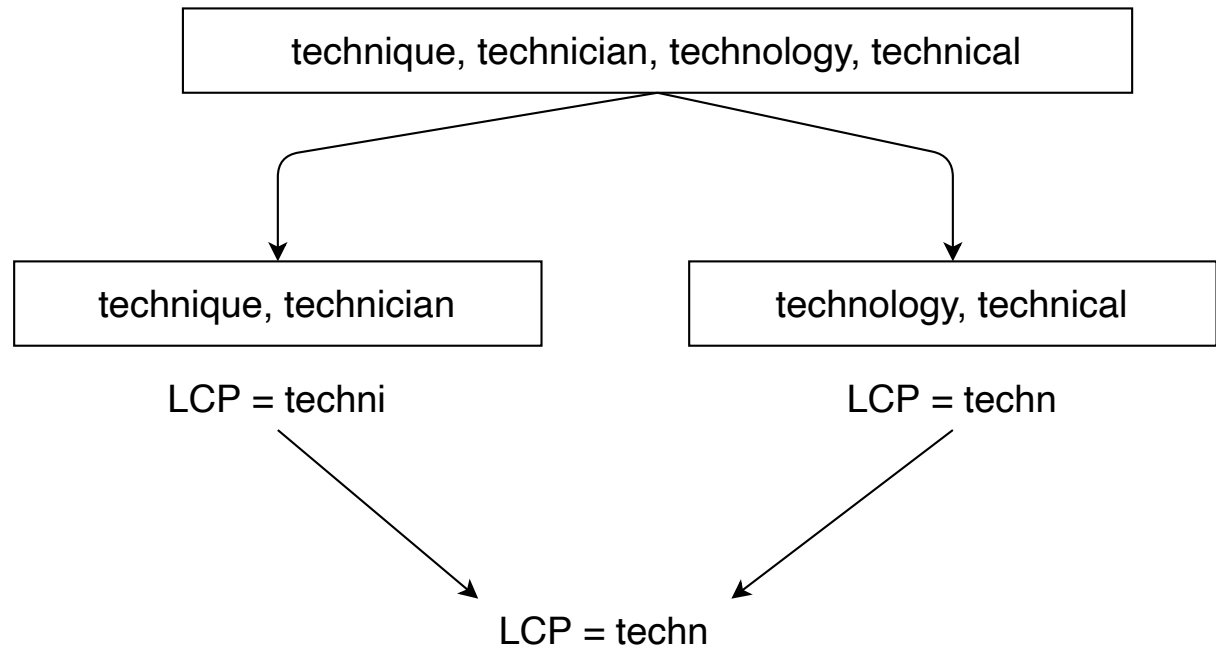
Esempio #2: Longest Common Prefix

Il problema: trovare il prefisso comune più lungo dato un array di stringhe

L'algoritmo:

1. **Dividere** la sequenza lunga n in due sottosequenze lunghe $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$
2. **Calcolare** ricorsivamente il LCP delle due sottosequenze (a meno che non abbiano un solo elemento)
3. **Combinare** i LCP fino a trovare il LCP dell'intero array

Graficamente



Problema

Ordinamento a paletta (OI 2015)

<https://training.olinfo.it/#/task/paletta/statement>



Romeo ha di recente assistito ad un'avvincente performance di un cuoco acrobatico, che gestiva una imponente grigliata di braciole ribaltandole a gruppi di tre per mezzo di una apposita paletta. Questo evento gli ha ispirato l'idea del *paletta-sort*, una nuova interessante procedura di ordinamento.

Dato un vettore V contenente gli interi da 0 a $N-1$ (indicizzato da 0 a $N-1$), l'unica operazione ammessa nel *paletta-sort* è l'operazione *ribalta*. Questa operazione sostituisce tre elementi A, B, C consecutivi di V con i corrispondenti ribaltati C, B, A . Aiuta Romeo a capire se è possibile ordinare il vettore V , e in caso affermativo quante e quali operazioni *ribalta* sono sufficienti!

Esempi di input/output

input.txt	output.txt
5 2 0 4 3 1	-1
6 2 3 0 5 4 1	3

Esercizi per casa

1. Domino massimale (ricorsione)
<https://training.olinfo.it/#/task/domino/statement>
2. Mappa antica (ricorsione)
<https://training.olinfo.it/#/task/mappa/statement>
3. Lotteria di quadri (divide-et-impera)
https://training.olinfo.it/#/task/abc_quadri/statement
4. Sorting proximity (divide-et-impera)
https://training.olinfo.it/#/task/ois_proximity/statement